

Surviving Driver Bugs with Rex: Hosting Device-Driver Logic in Safe Kernel Extensions

SSLab, National Taiwan University

Project repository: [ntu-ssl/rex-with-driver](#)

June 2026

Abstract

Device drivers remain the dominant source of kernel crashes. Rust-for-Linux removes whole classes of memory-safety bugs from drivers, but it does not change the kernel’s failure model: a Rust panic in a driver — for example a failed bounds check — is funneled into `BUG()`, producing an oops and, on hardened configurations, a full kernel panic. Rex is a recent kernel extension framework that replaces eBPF’s in-kernel verifier with language-based safety: extensions are written in safe Rust and a lightweight in-kernel runtime catches Rust panics, unwinds the extension gracefully, and lets the kernel keep running. Rex, however, only protects code that runs on its own dispatcher stack, at existing eBPF hook points.

This paper presents a driver architecture that extends Rex’s protection boundary to device-driver logic. We introduce a *dispatch hook-point* pattern: a Rust-for-Linux driver is split into a panic-free, in-kernel **thin shim** that owns the hardware-facing and VFS-facing structure, and a Rex extension that owns the **driver logic**. The shim exposes a single dispatch function that the extension intercepts via kprobe; a small map-based commit protocol lets the shim distinguish “logic completed”, “logic panicked and was recovered by Rex”, and “no logic attached”, and translate each into an ordinary `errno`. We build three drivers on this pattern — a file-operation forwarding driver, a virtual network driver whose per-packet transmit verdicts are decided by a Rex program, and a recoverable character device — plus a control-experiment driver containing the identical bug in the driver body. With the bug inside the Rex boundary, a triggering `write(2)` returns `EIO` and the system keeps serving; with the bug in the driver body, the same input ends in a kernel panic. Along the way we identify and solve two practical obstacles to hooking Rust kernel code: kCFI type-id mismatches on indirect calls into Rust, and silently ignored return-value overrides on optimized kprobes.

1 Introduction

Operating-system kernels are extended in two very different ways. Loadable kernel modules — and device drivers in particular — run with full kernel privileges and no safety net; empirical studies have repeatedly found that drivers exhibit error rates several times higher than the core kernel [1], and a single programming error can take down the whole machine. eBPF extensions, in contrast, are admitted into the kernel only after static verification and so cannot crash it, but the verifier sharply limits what they may do, and its usability problems — the *language-verifier gap* — are well documented [2].

Two recent developments change this landscape. First, Rust-for-Linux [3] brought a memory-safe language to in-tree drivers: large classes of bugs (use-after-free, data races, type confusion) are now compile errors. Yet Rust does not change the **failure semantics** of the kernel. Safe Rust checks many properties at runtime — array indexing, integer conversion, slicing — and a failed check raises a Rust *panic*. Inside the kernel there is no unwinder; the Rust-for-Linux panic handler prints the panic message and calls `BUG()`. The result is an oops, a killed task that never receives a meaningful error

code, leaked driver resources, and — under `CONFIG_PANIC_ON_OOPS=y` or `oops=panic`, common in production fleets — an immediate, machine-wide kernel panic. In other words, Rust-for-Linux converts silent memory corruption into a **reliably detected but still fatal** event.

Second, Rex [4] demonstrated that kernel extensions can be made safe without an in-kernel verifier at all. Rex extensions are written in a safe subset of Rust; memory, type, and resource safety come from the language and from a trusted kernel crate, while the properties unsuited to static analysis — stack depth, termination, and crucially **exception handling** — are enforced by a small in-kernel runtime. When a Rex program panics, the Rex panic handler releases the kernel resources the program acquired, control transfers to an in-kernel *landingpad*, and the dispatcher unwinds back to the calling context as if the program had returned an error. The kernel survives.

This suggests an appealing syllogism: if Rust panics are exactly the failure mode that remains in Rust drivers, and Rex is a production-grade mechanism for **surviving** Rust panics in kernel context, then driver logic placed inside the Rex protection boundary should survive its own bugs. There is a catch, however. Rex’s runtime only protects code launched through its dispatcher — in practice, programs attached to existing eBPF hook points such as kprobes, tracepoints, and XDP. A device driver is not an eBPF hook point. Its entry points are called directly by the VFS and the networking core, on the regular kernel stack, where a panic still means `BUG()`.

Our approach. We invert the usual relationship between extensions and the kernel. Instead of using Rex to observe a driver, we restructure the driver so that its *logic* runs **as** a Rex extension:

- The in-kernel part of the driver shrinks to a **thin shim**, written in Rust-for-Linux, that registers the device and forwards every interesting event (a file operation, a packet transmission) to one exported, never-inlined dispatch function with a fixed C ABI. The shim is small, has no interesting control flow, and is engineered to be panic-free.
- The driver **logic** — protocol handling, state machines, policy — lives in a Rex program that attaches to the dispatch function with a kprobe. It runs on Rex’s dedicated per-CPU stack, under Rex’s panic handler, stack monitor, and termination watchdog.
- A small **commit protocol** over shared eBPF maps lets the shim’s dispatch body learn the outcome: a committed result (success), an in-flight marker without a result (the logic panicked and Rex recovered it), or neither (no logic attached). Each outcome maps to an ordinary return value or `errno`.

The net effect is that the same bug which would crash the machine inside a driver body instead surfaces to userspace as `write(2) = -1, errno = EIO`, followed by normal operation.

Contributions. This work makes the following contributions:

1. A reusable *dispatch hook-point* pattern for making Rust-for-Linux driver code interceptable and controllable by Rex programs, including the exact set of attributes (`#[no_mangle]`, `#[inline(never)]`, `extern "C"`, `core::hint::black_box`) and kernel configuration (`CONFIG_KPROBE_EVENTS_ON_NOTRACE=y`) required to make a Rust function a reliable kprobe target (§3).
2. Three drivers built on the pattern: `fwd_driver`, which forwards all file operations of a character device to a Rex program that decides their outcomes; `net_driver`, a virtual Ethernet device whose transmit datapath is filtered per-packet by a Rex program; and `recover_driver`, whose entire write/read logic executes inside the Rex boundary and survives an injected out-of-bounds bug (§4).
3. A matched control experiment, `baseline_driver`, containing the identical logic and identical bug in the driver body, demonstrating the difference in failure semantics end to end: `errno=EIO` and continued service versus a kernel panic (§5).
4. Two practical findings about hooking Rust kernel code that we believe are independently useful: (a) under kCFI, Rust functions cannot be stored directly in C operation tables whose signatures involve

C enums — a thin clang-compiled trampoline is required (§4.2); and (b) `bpf_override_return` is silently ignored on *optimized* kprobes, which motivates our map-based “kernel pull” commit protocol as the robust alternative (§3.3, §4.3).

2 Background

2.1 eBPF and the language-verifier gap

eBPF is the de facto kernel extension mechanism in Linux [5]. Its safety story rests on an in-kernel verifier that symbolically executes extension bytecode before loading, checking memory safety, type safety, resource release, termination, and stack bounds. The verifier’s static nature is also its weakness: safe programs are routinely rejected because they exceed verification-complexity limits, because LLVM emitted correct-but-unexpected bytecode, or because of verifier bugs. Jia et al. analyzed 72 such incidents across Cilium, Katran, and Aya and catalogued the workarounds developers are forced into — splitting programs, inline-assembly pinning of values, reimplementing `memcpy` [4]. The position paper accompanying that line of work argues that verification of extensions at the bytecode level is fundamentally untenable as extensions grow [2]. BMC, an in-kernel Memcached cache, had to be split into seven tail-called programs to satisfy the verifier [6]. KFlex extends eBPF expressiveness with software fault isolation but inherits the verifier and hence the gap [7].

2.2 Rex

Rex [4] closes the language-verifier gap by dropping the verifier entirely. A Rex extension is written in **safe Rust only** (unsafe code, `mem::forget`, floating point, and similar features are rejected by the toolchain) against a trusted **kernel crate** that wraps the eBPF helper interface and maps in safe Rust APIs. Properties that resist static analysis are enforced by a small in-kernel runtime:

- **Graceful exception handling.** The Rex dispatcher (`rex_dispatcher_func`) saves the kernel stack pointer in per-CPU storage, switches to a dedicated per-CPU extension stack, and calls the program. On a Rust panic, Rex’s panic handler releases any kernel resources recorded for the program, then jumps to an in-kernel **landingpad** that logs the panic, installs a default error return value, and resumes at the dispatcher’s exit path, restoring the original stack pointer. To the kernel, the program appears to have returned an error.
- **Stack safety** via static stack-usage computation where possible and compiler-inserted runtime checks otherwise, on the dedicated 8-page stack.
- **Termination** via per-CPU hrtimer watchdogs that redirect a long-running program into the panic path.

Rex programs attach at existing eBPF hook points; the program type used throughout this paper is the **kprobe** program, which receives the probed function’s register file (`pt_regs`) and may call `bpf_override_return` to skip the probed function and force its return value [8].

The crucial boundary condition for our work: Rex protects **only** code that entered through its dispatcher. The framework makes the kernel safe from the extension; it says nothing about driver code in module context.

2.3 Rust-for-Linux and the panic problem

Rust-for-Linux [3] provides abstractions (`miscdevice`, `module!`, `Kiocb`/iov iterators, etc.) for writing in-tree drivers in Rust. Language safety eliminates spatial and temporal memory errors in safe code, but **logic** errors that trip runtime checks remain — and their handling is draconian. The kernel’s Rust panic handler prints the message and calls `BUG()`:

```
rust_kernel: panicked at rex_baseline_main.rs:97:
            index out of bounds: the len is 8 but the index is 10
```

```

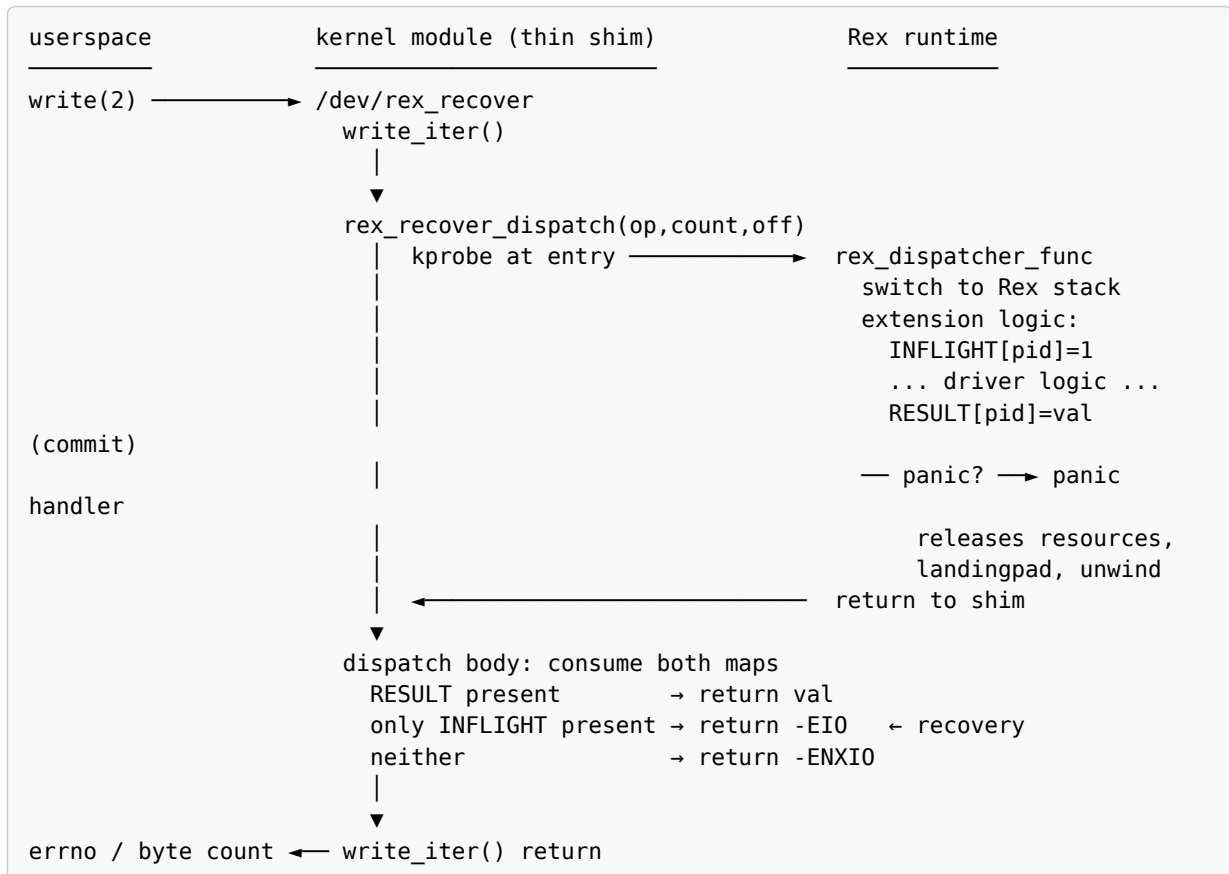
kernel BUG at rust/helpers/bug.c:7!
Oops: invalid opcode: 0000 [#1] SMP
...
Kernel panic - not syncing: Fatal exception

```

Even without `CONFIG_PANIC_ON_OOPS`, the writing process is killed with no `errno` semantics, the kernel is tainted, locks held at the panic site stay locked, and driver resources leak. The driver cannot catch its own panic: in-kernel Rust is compiled with `panic=abort` and there is no unwinder.

3 Design: the dispatch hook-point pattern

Our goal is to route driver control flow through Rex’s protection boundary with the smallest possible trusted residue inside the kernel. The pattern has three parts: a hookable dispatch function (§3.1), an extension that attaches to it (§3.2), and a result-passing protocol (§3.3). Listing 1 shows the overall flow for the recoverable driver.



Listing 1: Control and result flow in `recover_driver`. The bug-prone driver logic runs on the Rex stack, where a Rust panic is caught by Rex’s landingpad; the in-kernel shim infers the panic from the absence of a committed result and converts it into `-EIO`.

3.1 A Rust function as a reliable kprobe target

The shim exposes exactly one extension point per event class, e.g.:

```

#[no_mangle]
#[inline(never)]
pub extern "C" fn rex_fwd_dispatch(op: u64, count: u64, offset: u64) -> i64 {
    core::hint::black_box((op, count, offset));
}

```

```
core::hint::black_box(DISPATCH_UNHANDLED) // -ENOSYS: "no extension loaded"
}
```

Each attribute is load-bearing:

- `#[no_mangle]` gives the function a stable kallsyms-resolvable symbol so the kprobe can be attached by name.
- `#[inline(never)]` guarantees a real call site and entry point exist — an inlined hook point silently never fires.
- `extern "C"` pins the argument registers: `op`, `count`, `offset` arrive in `rdi/rsi/rdx`, exactly where a kprobe program reads them from `pt_regs`.
- `core::hint::black_box` keeps the arguments observably live at entry and makes the return value opaque, so the optimizer can neither dead-code the marshaling nor constant-propagate the default `-ENOSYS` into callers — callers must read `rax`, which is where an override lands.

One kernel-configuration change is required. Rust kernel functions are not instrumented for ftrace, and by default the kprobe-events infrastructure refuses probes on such `notrace` functions. We build the kernel with `CONFIG_KPROBE_EVENTS_ON_NOTRACE=y`, after which a plain `int3` kprobe on the Rust symbol works. Notably, return-value override does **not** require ftrace (`__fentry__`): `bpf_override_return` works on an ordinary kprobe by rewriting `regs->ip` from the pre-handler — with the important exception discussed next.

3.2 The extension side

The driver logic is an ordinary Rex kprobe program:

```
#[rex_kprobe(function = "rex_net_dispatch")]
fn net_filter(obj: &kprobe, regs: &mut PtRegs) -> Result {
    let (op, len, proto) = (regs.rdi(), regs.rsi(), regs.rdx());
    ...
    obj.bpf_override_return(regs, VERDICT_DROP); // or VERDICT_PASS
    Ok(0)
}
```

Because the program is a Rex program, it inherits the entire Rex safety contract for free: it cannot perform unsafe memory access, it cannot leak the kernel resources it acquires, it cannot overflow the kernel stack, it cannot loop forever — and, the property this paper leans on, **a panic inside it is survivable**.

3.3 Returning results: push vs. pull

The extension must communicate a result back to the shim. We use two mechanisms with different robustness/effort trade-offs.

Push: `bpf_override_return`. The extension rewrites the dispatch function’s return value; the shim just reads its own call’s result. This is the simplest protocol and is used by `fwd_driver` and `net_driver`. It has a sharp edge: when the kernel *optimizes* a kprobe (replacing the `int3` with a jump, “optprobe”), the trampoline does not re-load `regs->ip` after the pre-handler, so the override is **silently ignored** and the caller sees the default `-ENOSYS`. Our tests disable optimization first (`echo 0 > /proc/sys/debug/kprobes-optimization`); a production deployment would either do the same for these probes or use the pull protocol.

Pull: map-based commit. `recover_driver` does not use override at all. The extension and the shim share two pinned maps keyed by `pid`:

State observed by dispatch body	Meaning	Return
RESULT[pid] present	logic ran to completion; the value is the committed return	RESULT[pid]
only INFLIGHT[pid] present	logic started but never committed — it panicked and Rex recovered it	-EIO
neither present	no extension attached	-ENXIO (fail-closed)

Table 1: The three-state commit protocol. Both entries are consumed (deleted) on every dispatch, so stale state cannot leak into the next operation.

The protocol’s correctness rests on a single ordering rule, *commit last*: writing **RESULT[pid]** is the final action of the success path, so any panic necessarily happens before the commit, and “INFLIGHT without RESULT” is a sound panic detector — the kernel never needs to tell the shim that a panic occurred. The same ordering yields state consistency: the extension’s internal state updates (its **TOTAL** accumulator) are placed **after** the panic-prone computation and **before** the commit, so a panicked operation contributes no partial state. Because the pull protocol never touches **regs->ip**, it is immune to kprobe optimization.

3.4 Failure policy

Rex’s default failure model is crash-stop: after catching a panic it sends **SIGSYS** to the process that loaded the extension; an unhandled **SIGSYS** kills the loader, closing the bpf link and detaching the program, after which every operation returns **-ENXIO**. This is a reasonable fail-stop default, but our recoverable driver wants “panic, report, **keep serving**”. The loader therefore installs a real **SIGSYS** handler — **SIG_IGN** is insufficient, because the kernel delivers the signal via **force_sig**, which resets ignored dispositions to default. Whether a panicked program should be killed, kept, or rate-limited is naturally a per-program policy; we return to this in §7.

4 The drivers

All four drivers follow Rust-for-Linux idioms (the **module!** macro, **MiscDevice** with **read_iter/write_iter** on **Kiocb**, RAII registration) and build out of tree against the Rex kernel with **make LLVM=1**.

4.1 fwd_driver: file operations under extension control

fwd_driver is the minimal instantiation of the pattern. The shim registers **/dev/rex_fwd** and forwards all four file operations — open, release, read, write — through **rex_fwd_dispatch(op, count, offset)**. With no extension loaded, the sentinel **-ENOSYS** return makes the device safely inert (opens succeed, reads return 0 bytes, writes are swallowed). The companion extension demonstrates both **monitoring** and **control**: it keeps a hash map of PIDs that currently hold the device open, records every operation into a 64-entry ring of **FwdEvent** structs (a **#[repr(C)]** plain-integer struct, so userspace can read the map without unsafe code), and uses **bpf_override_return** to decide each operation’s outcome — e.g., claiming all bytes of a write so the caller does not retry, or rejecting an open with **-EACCES**.

4.2 net_driver: a Rex-controlled network datapath

net_driver ports the pattern from file operations to the network transmit path. The shim registers a virtual Ethernet interface **rexnet0** (**alloc_netdev_mqs + register_netdev** via Rust bindings we added for **netdevice.h/etherdevice.h**); its **ndo_start_xmit** forwards every outgoing packet’s length and EtherType to **rex_net_dispatch(OP_XMIT, len, proto)** and interprets the verdict: non-negative or **-ENOSYS** → transmit (counted **tx_pass**), other negatives → drop (counted **tx_drop**).

The counters live in the **driver**, so Rex’s effect on the datapath is observable independently of the extension’s own statistics. The sample extension counts packets and bytes, tallies per-protocol totals in an array map, passes IPv4, and drops ARP.

Two findings make this driver more than a transplant of **fwd_driver**:

kCFI and Rust `ndo` callbacks. The kernel is built with kCFI (`CONFIG_CFI=y`) [9]. A netdev’s `ndo_start_xmit` is invoked *indirectly* through a C function pointer, and the call site verifies the target’s kCFI type id. `ndo_start_xmit` returns `netdev_tx_t` — a C enum — which bindgen flattens to `c_int` on the Rust side; rustc therefore computes the type id of a function returning `int`, clang expects the id for the `enum` return, and the indirect call traps with an invalid-opcode oops. The fix is a 60-line clang-compiled glue file: the `net_device_ops` table points at C trampolines (correct kCFI ids), which forward to the Rust logic with **direct** calls, which kCFI does not check. **fwd_driver** never encountered this because its hook point is only ever called directly. We expect this trampoline pattern to recur wherever Rust logic must live behind C operation tables under kCFI.

Optprobes vs. override. On this hot, repeatedly-fired probe the kernel aggressively converts the `int3` kprobe into an optprobe, which silently drops `bpf_override_return`’s `regs->ip` rewrite (§3.3). The in-VM test disables kprobe optimization before attaching. This experience is what motivated designing **recover_driver**’s protocol around the kernel-pull model instead.

4.3 **recover_driver** and **baseline_driver**: the recovery experiment

recover_driver is the end-to-end demonstration that driver logic inside the Rex boundary survives its own bugs. The shim (`/dev/rex_recover`) contains **no** logic: `write_iter/read_iter` call `rex_recover_dispatch`, whose body collects the three-state outcome of Table 1 via a small C component (`rex_recover_shim.c`) in the same module — Rust-for-Linux has no BPF-map bindings, so map lookup/delete against `struct bpf_map` is delegated to C, following the `.rs + .c` single-module layout of the in-tree `rust_print` sample. The loader passes the two map fds to the shim with two `ioctls`; the shim takes references via `bpf_map_get(fd)` and validates key/value sizes.

The extension implements the “driver logic”: a write of n bytes is classified into one of eight weight classes (`class = n / 4`) through a fixed 8-entry lookup table, a weighted total is accumulated in a `TOTAL` array map, and the operation commits `RESULT[pid] = n`. The lookup carries a deliberately preserved bug — the developer “assumed” single writes are under 32 bytes, so any `write(2)` of $n \geq 32$ bytes computes `class >= 8` and the array index panics (**black_box** keeps the bounds check at runtime).

baseline_driver is the matched control: the **identical** classification logic, table, and bug, written line-for-line in the driver’s own `write_iter` on `/dev/rex_baseline`, with no Rex involvement. The two drivers are exercised by the same write scenario.

5 Evaluation

Our evaluation is functional: it asks whether the architecture delivers the promised failure semantics, end to end, on a real kernel. All experiments run in QEMU VMs booted from the project’s Rex kernel (Linux 6.19 with the Rex runtime, `CONFIG_RUST=y`, kCFI enabled, `CONFIG_PANIC_ON_OOPS=y`, and the q-script harness additionally passing `oops=panic`), using the repository’s automated guest tests.

5.1 Panic containment: **recover** vs. **baseline**

Both drivers receive the same scenario: writes of 5, 17, 40, and 9 bytes, then a 16-byte read. The 40-byte write trips the out-of-bounds bug.

Operation	recover_driver (bug in Rex ext.)	baseline_driver (bug in driver body)
write 5 B	ret=5	ret=5
write 17 B	ret=17	ret=17
write 40 B	errno=5 (EIO) — kernel alive	BUG() → oops → kernel panic
write 9 B	ret=9	— (machine is down)
read 16 B	ret=0 (EOF)	—

Table 2: The motivating result: identical logic, identical bug, identical input — opposite failure semantics depending on which side of the Rex boundary the logic runs.

On the recover side, the console shows Rex’s landingpad report (**rex: Panic from Rex prog: index out of bounds: the len is 8 but the index is 10**) followed by the shim’s **rex_recover: logic panicked mid-operation; recovered, returning -EIO**, and the subsequent 9-byte write and the read succeed normally — the panicked operation contributed nothing to the **TOTAL** accumulator, as guaranteed by the commit-last ordering. On the baseline side the same input produces the oops transcript of §2.3 and the machine never executes another instruction of the test. The baseline test is deliberately excluded from the automated meson suite: its expected outcome is that the VM dies.

5.2 The network datapath

The **net_driver** guest test loads the shim, brings up **rexnet0** with an address, attaches the extension, and generates traffic: ICMP echo requests (passed), ARP requests (dropped by policy), and one ICMP packet padded to exactly 1000 bytes — the length the extension uses to index a 4-entry table, triggering a bounds-check panic **in the middle of the transmit datapath**. The test passes when (a) the driver’s own **tx_pass/tx_drop** counters match the extension’s map-side counters for the policy traffic, (b) the panic is caught by Rex (landingpad report in dmesg, packet falls through to the driver’s default PASS), and (c) the interface continues to transmit afterwards, finishing with **NET_RESULT=success**. This shows the recovery property is not an artifact of the slow character-device path: a per-packet hook with a verdict protocol survives a mid-datapath panic.

5.3 Microtests of the exception-handling boundary

Two smaller samples bracket the system tests. **panic_test** attaches to a trivial module-provided target and drives it with an **ioctl**-supplied index into a 4-entry table, giving a one-knob way to flip between the normal path and a Rex-caught panic; it confirms the landingpad fires and the system survives at the smallest possible scale. The **test** sample exercises the monitoring half of the pattern on unmodified kernel code, attaching Rex kprobes to **vfs_read/vfs_write** and a tracepoint to observe file operations on a target device. Together with **key-filter-test** (a tracepoint program filtering input events) these validate that the Rex toolchain, loader, maps, and trace output behave as expected in our kernel before the driver experiments are layered on top.

5.4 Trusted residue

The pattern’s value depends on the shim staying small and boring. The **recover_driver** shim is 201 lines of Rust plus 136 lines of C map glue; the **net_driver** shim is 218 lines of Rust plus a 60-line kCFI trampoline file. Neither contains a loop, an array index, or arithmetic that can panic; the Rust portions use only infallible Rust-for-Linux idioms on their hot paths. We do not claim these shims are verified — they are the same kind of trusted code as Rex’s own kernel crate, just much smaller than the logic they host.

6 Relation to Rex

This project is best read as a consumer — and a stress test — of Rex’s central design decisions [4]:

The exception-handling runtime is the enabling mechanism. Everything in §4.3 reduces to Rex’s guarantee that a panic in extension context resets the stack, releases recorded resources, and returns control with a default error. The Rex paper presents this as a safety property (panics must not crash the kernel); we use it as a **recovery** primitive (panics become `errno`s). The commit protocol of Table 1 is precisely the adapter between Rex’s “program vanished mid-flight” semantics and a driver’s “every operation returns a value” contract.

Closing the language-verifier gap is what makes driver logic expressible. Driver logic — state machines, accumulators, lookup tables, multi-step protocols — is exactly the kind of code the eBPF verifier punishes (§2.1) [2]. Writing the same logic as safe Rust against Rex’s maps and helpers required no verifier appeasement: the buggy classification logic in `recover_driver` is seven straight-line lines.

We extend Rex’s reach, not its TCB. Rex attaches at existing eBPF hook points; drivers are not among them. The dispatch hook-point pattern manufactures a Rex-compatible hook point inside a driver using only stock mechanisms — a kprobe on an exported Rust symbol and ordinary eBPF maps. The Rex runtime, kernel crate, and toolchain are used unmodified; our kernel-side changes are limited to a config flag (`CONFIG_KPROBE_EVENTS_ON_NOTRACE=y`) and Rust-for-Linux **bindings** (netdevice helpers), not the Rex framework itself.

We inherit Rex’s trade-offs. The logic is confined to Rex’s safe interface: no arbitrary kernel calls, no sleeping, no direct hardware access. And we surface one place where Rex’s policy is currently one-size-fits-all: the crash-stop `SIGSYS` default (§3.4) is right for observability programs but wrong for a driver that should degrade to `-EIO` and keep serving; a per-program panic policy in the kernel would remove the need for the loader’s signal handler.

7 Limitations and future work

This is not transparent driver rescue. The logic must be (re)written as a Rex extension; existing drivers are not saved by recompilation. The pattern fits *logic-type* drivers — protocol handling, state machines, policy, software devices — and not drivers whose essence is touching hardware registers, since the Rex interface (deliberately) does not expose them. A “route B” that retrofits protection onto existing driver bodies would need a different mechanism.

Data movement is incomplete. The read path currently returns EOF or a length without filling the user buffer; delivering bytes from the extension requires `bpf_probe_write_user` (push) or shim-mediated buffers (pull) and is the most immediate piece of future work.

Protocol edge cases. If inserting `INFLIGHT[pid]` itself fails (map full), a subsequent panic is misclassified as “no extension” (`-ENXIO` instead of `-EIO`) — fail-closed, but with the wrong `errno`. Keying by PID assumes the dispatch body and the kprobe handler run in the same task, which holds for synchronous file operations but would need rethinking for asynchronous contexts.

Overhead is unmeasured. Each operation pays a kprobe trap (un-optimized, in the override variant), a dispatcher stack switch, and two map operations. The Rex paper reports its dispatcher and runtime checks are competitive with eBPF on network fast paths [4], and none of our protocol adds more than constant work, but quantifying the per-operation cost — `int3` kprobe vs. `optprobe` vs. a future direct dispatch — is necessary before claiming the pattern for hot datapaths.

Hook-point integrity. The shim trusts that **some** Rex program of the right shape is attached; it cannot tell which. Binding a dispatch symbol to a specific extension identity (e.g., verifying the attached program at `ioctl` registration time) would harden the design.

8 Related work

Driver fault isolation has a long history. Nooks isolates drivers in in-kernel protection domains with shadow copies and recovery agents [10]; SafeDrive adds language-level annotations and recovery to C drivers [11]. These systems retrofit protection onto existing C drivers at the cost of substantial kernel mechanism; our pattern instead **relocates** logic into an extension framework that already provides the safety properties, keeping the kernel-side mechanism to a kprobe and two maps. Microkernels solve the problem by construction, running drivers in user space, with the attendant IPC costs. On the extension side, eBPF [5] and KFlex [7] provide verified or SFI-confined execution at fixed hook points, but the verifier limits the expressible logic (§2.1); Rex [4] supplies the safety foundation we build on. Rust-for-Linux [3] removes memory unsafety from drivers but, as our baseline shows, retains fail-stop semantics for runtime check failures; our work can be seen as composing the two — RfL for the structure, Rex for the logic — so that the remaining failure mode of safe Rust becomes recoverable.

9 Conclusion

A Rust panic in a driver body is a kernel panic; the same panic inside Rex’s protection boundary is an **errno**. This paper showed that the boundary can be moved to enclose device-driver logic with surprisingly little mechanism: one exported, never-inlined dispatch function per driver, a kprobe, two maps, and a commit-last protocol. Across a character device and a network datapath, the pattern let deliberately buggy logic be monitored, controlled, and — when it panicked — recovered, while a line-for-line identical baseline took the machine down. The experiments also charted the practical terrain of hooking Rust kernel code under modern hardening: kCFI demands clang trampolines at C operation tables, and optprobes silently defeat return-value overrides, favoring pull-style result protocols. We see this as a first concrete step from “safe kernel extensions” toward “kernel components that survive their own bugs”, with data delivery, per-program failure policy, and a quantified-overhead datapath as the immediate next steps.

References

- [1] A. Chou, J. Yang, B. Chelf, S. Hallem, and D. Engler, “An Empirical Study of Operating System Errors,” in *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP '01)*, 2001.
- [2] J. Jia, R. Sahu, A. Oswald, D. Williams, M. V. Le, and T. Xu, “Kernel extension verification is untenable,” in *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS XIX)*, 2023.
- [3] Rust for Linux contributors, “Rust for Linux.” 2025.
- [4] J. Jia *et al.*, “Rex: Closing the language-verifier gap with safe and usable kernel extensions,” in *Proceedings of the 2025 USENIX Annual Technical Conference (ATC '25)*, 2025.
- [5] eBPF community, “eBPF documentation.” 2025.
- [6] Y. Ghigoff, J. Sopena, K. Lazri, A. Blin, and G. Muller, “BMC: Accelerating Memcached using Safe In-kernel Caching and Pre-stack Processing,” in *Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI '21)*, 2021.
- [7] K. K. Dwivedi, R. Iyer, and S. Kashani, “Fast, Flexible, and Practical Kernel Extensions,” in *Proceedings of the 30th Symposium on Operating Systems Principles (SOSP '24)*, 2024.
- [8] Linux kernel documentation, “Kernel Probes (Kprobes).” 2025.

- [9] Linux kernel documentation, “Control Flow Integrity (kCFI) in the Linux kernel.” 2025.
- [10] M. M. Swift, B. N. Bershad, and H. M. Levy, “Improving the Reliability of Commodity Operating Systems,” in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, 2003.
- [11] F. Zhou *et al.*, “SafeDrive: Safe and Recoverable Extensions Using Language-Based Techniques,” in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*, 2006.